

# Simple Datatypes - Questions

---

## Question: Binary Numbers

**Short Humorous Scenario:** Martin har fundet en gammel computer fra 1980'erne, der kun kan tælle til 7. Han er forvirret, for han ved, at computeren burde kunne tælle højere. Han har bedt jer om at finde ud af, hvorfor den har denne begrænsning. *"Måske bruger den bare 3 bit? Kan I regne det ud?"*

### Hvad de allerede ved:

- Binary number system (base-2)
- Bit representation: 0 og 1
- Decimal number system (base-10)

### Opgaven:

1. Beregn: Hvor mange unikke tal kan repræsenteres med et 3-bit binært tal?
2. Beregn: Hvad er det højeste tal (i base-10) der kan repræsenteres med et 3-bit binært tal?
3. Vis dit arbejde
4. **PBL:** Martin spørger: *"Hvad ville svaret være for et 4-bit binært tal?"*

### Hint:

- Hvert bit kan være enten 0 eller 1
- Med n bits har du  $2^n$  mulige kombinationer
- For 3 bits:  $2^3 = 8$  kombinationer
- Det højeste tal er  $2^n - 1$
- List alle 3-bit kombinationer for at verificere

### Hvad du aldrig må gøre:

- Tæl fra 0 til 9 (det er decimal, ikke binær)
- Glem at 000 også er en gyldig kombination
- Forveksle bit antal med det faktiske værdiområde

---

## Question: ASCII Characters

**Short Humorous Scenario:** Martin har modtaget en mystisk besked med hexadecimale tal: 0x55, 0x21, 0x33. Han har ingen anelse om, hvad de betyder. *"Kan I ikke finde ud af, hvilke tegn disse tal repræsenterer?"* han spørger desperat.

|   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | A   | B   | C  | D  | E  | F   |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|-----|----|----|----|-----|
| 0 | NUL | SOH | STX | ETX | EOT | ENQ | ACK | BEL | BS  | HT | LF  | VT  | FF | CR | SO | SI  |
| 1 | DLE | DC1 | DC2 | DC3 | DC4 | NAK | SYN | ETB | CAN | EM | SUB | ESC | FS | GS | RS | US  |
| 2 | SPC | !   | "   | #   | \$  | %   | &   | '   | (   | )  | *   | +   | ,  | -  | .  | /   |
| 3 | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | :   | ;   | <  | =  | >  | ?   |
| 4 | @   | A   | B   | C   | D   | E   | F   | G   | H   | I  | J   | K   | L  | M  | N  | O   |
| 5 | P   | Q   | R   | S   | T   | U   | V   | W   | X   | Y  | Z   | [   | \  | ]  | ^  | _   |
| 6 | `   | a   | b   | c   | d   | e   | f   | g   | h   | i  | j   | k   | l  | m  | n  | o   |
| 7 | p   | q   | r   | s   | t   | u   | v   | w   | x   | y  | z   | {   |    | }  | ~  | DEL |

### Hvad de allerede ved:

- ASCII character encoding
- Hexadecimal notation (0x prefix)
- `printf` funktion med format specifiers
- Character literals i C

### Opgaven:

1. Slå ASCII-tegnene op for disse hexadecimale værdier:

- 0x55
- 0x21
- 0x33

2. Lav et program der udskriver hvert tegn ved hjælp af `printf`:

- Brug format: `printf("0x55: %c\n", 0x55);`

3. Kør programmet og verificer outputtet

4. **PBL:** Martin spørger: "Hvad sker der, hvis I udskriver disse som heltal ved at bruge `%d` i stedet for `%c`? Prøv det og forklar forskellen."

### Hint:

- 0x55 = 85 i decimal
- 0x21 = 33 i decimal
- 0x33 = 51 i decimal
- Brug en ASCII-tabel til at finde tegnene
- `%c` udskriver et tegn, `%d` udskriver et decimaltal

### Hvad du aldrig må gøre:

- Brug string literals som "U" - brug karakterkoder
- Glem at inkludere `stdio.h`
- Forveksle `%c` og `%d` format specifiers

---

## Question: ESP32 Unicode

**Short Humorous Scenario:** Martin har fået en ESP32 til at kommunikere med nogle mystiske tegn. Han har ingen anelse om, hvilket sprog det er. *"Der står noget med 'Size of string' og nogle mærkelige symboler. Kan I ikke hjælpe mig med at finde ud af, hvilket sprog det er?"*

**Hvad de allerede ved:**

- `sizeof` operator
- Character arrays og strings
- Unicode character encoding
- ESP32 Arduino IDE
- `Serial.print()` / `Serial.println()`

**Opgaven:**

1. Åbn og kørs programmet `files/esp32_unicode/esp32_unicode.ino` på din ESP32
2. Observer outputtet i Serial Monitor
3. For hver string output, brug Google Translate til at identificere:
  - a. Sproget
  - b. Oversættelsen til dansk
4. Skriv resultaterne ned
5. **PBL:** Martin spørger: *"Hvorfor tager nogle tegn mere bytes end andre? Hvad er forskellen mellem ASCII og Unicode?"*

**Hint:**

- Brug <https://translate.google.com> til oversættelser
- Noter byte-størrelsen der bliver printet før hver string
- Unicode bruger flere bytes til ikke-ASCII tegn
- ASCII bruger 1 byte per tegn (7 bits + 1 parity bit)

**Hvad du aldrig må gøre:**

- Gæt sproget uden verifikation
- Ignorer byte-størrelsesinformationen
- Glem at køre programmet på ESP32

---

## Question: Patient Data Types

**Short Humorous Scenario:** Martin skal oprette et patientsystem, men han ved ikke, hvilke datatyper han skal bruge. *"Jeg har alle disse patientoplysninger, men hvordan gemmer jeg dem korrekt? Og hvordan beregner jeg BMI?"* han spørger forvirret.

**Hvad de allerede ved:**

- Variabel declaration: `int`, `char`, `float`, `double`
- Comments i C: `//` og `/* */`
- Arithmetic operators: `/`, `*`
- `printf` med format specifiers: `%d`, `%c`, `%f`, `%s`
- Casting mellem typer

### Opgaven:

1. Lav et program kaldet `patient.c` der kan gemme variabler for en patient med:
  - id (heltal)
  - age (heltal)
  - gender ('M'/'F')
  - marital\_status ('G'/'S') hvor G = gift, S = single
  - weight (i kg, ikke-heltal)
  - height (i meter, ikke-heltal)
2. Tilføj passende kommentarer der forklarer dine datatype valg
3. Udskriv alle variablers navne og deres værdier til terminalen
4. Hvis gender er 'F', udskriv "Kvinde"; hvis 'M', udskriv "Mand"
5. Hvis marital\_status er 'G', udskriv "Gift"; hvis 'S', udskriv "Single"
6. Beregn og udskriv BMI ved hjælp af: `bmi = weight / (height * height)`
7. **PBL:** Martin spørger: "Kan I udvide programmet til at udskrive 'Underweight' hvis BMI < 18.0, 'Normal weight' hvis BMI er mellem 18.0 og 25.0, og 'Overweight' hvis BMI >= 25.0?"

### Hint:

- Brug `int` til id, age
- Brug `char` til gender og marital\_status
- Brug `float` eller `double` til weight og height (ikke-heltal værdier)
- Brug `if-else if-else` struktur til BMI klassifikation
- Husk: height \* height betyder height i andet (kvadreret)

### Hvad du aldrig må gøre:

- Brug forkerte datatyper (f.eks. int til weight)
- Glem at udskrive alle variabelværdier
- Glem at inkludere beregningskommentarer
- Hardcode BMI kategorierne uden at bruge variabler

---

## Question: Binary til Decimal

**Short Humorous Scenario:** Martin har set nogle tal skrevet med 0b- og 0x-præfikser, og han forstår dem slet ikke. "Hvad betyder disse 0b1111 og 0xf? Kan I ikke lave et program, der kan fortælle mig, hvad de egentlig er?"

**Hvad de allerede ved:**

- Integer literals: decimal, binary (0b), hexadecimal (0x)
- Variabel declaration og initialisering
- `printf` med format specifiers
- Type conversion

**Opgaven:**

1. Lav et program der udskriver den decimale (base-10) repræsentation af:
  - Binært: `0b1111`
  - Hexadecimalt: `0xf`
2. Deklarer en integer `a` og sæt den til `0b1111`
3. Udskriv `a` til terminalen
4. Kør programmet og observer outputtet
5. **PBL:** Martin spørger: "Hvad er den decimale værdi af `0b10101010`? Beregn det manuelt først, derefter verificer med et program."

**Hint:**

- Binært `0b1111` =  $18 + 14 + 12 + 11 = 15$
- Hexadecimalt `0xf` = 15 i decimal
- Compileren konverterer automatisk mellem baser
- Du kan deklare: `int a = 0b1111;`
- Til manuel beregning: hver bit position er en potens af 2

**Hvad du aldrig må gøre:**

- Udskriv den bogstavelige string "0b1111" i stedet for værdien
- Glem at 0b præfiks betyder binær
- Glem at 0x præfiks betyder hexadecimal

---

## Question: Decimal til Binary (Hard)

**Short Humorous Scenario:** Martin har hørt, at computeren forstår binære tal, men han forstår ikke, hvordan man konverterer fra decimal til binær. "Jeg har brug for at se, hvordan tallene ser ud i computerens verdens! Kan I lave et program, der kan vise mig det?"

**Hvad de allerede ved:**

- Binary number system
- Bitwise operators: `&`, `>>`, `<<`
- Loops (`for`, `while`)
- `printf` funktion
- `sizeof` operator
- Integer size: `int_sz_bits = sizeof(int) * 8`

## Opgaven:

1. **Manuel beregning først:** Skriv den 4-bit binære repræsentation for disse tal (i kommentarer i dit program):
  - int: 1
  - int: 2
  - int: 8
2. Lav et program kaldet `tal2bin.c`
3. Deklarer en integer `tal = 10`
4. Lav en unsigned integer `mask = 1 << (sizeof(int) * 8 - 1)`
5. For hver bit i `tal` (loop med iterator `i`):
  - a. Test om `tal & mask` evaluerer til true, udskriv "1", ellers udskriv "0"
  - b. Right-shift masken: `mask = mask >> 1`
6. Loop gennem integer fra -15 til 15 og udskriv deres binære repræsentation
7. **PBL:** Martin spørger: *"Kan I modificere programmet til også at vise den hexadecimale repræsentation ved siden af den binære?"*

## Hint:

- Hvornår evaluerer en værdi til TRUE vs FALSE i C? (non-zero = TRUE)
- `sizeof(int) * 8` giver antallet af bits i en int
- `1 << n` skaber en bit maske med 1 på position n
- Right-shifting masken flytter 1-bitten til højre
- Brug `%x` format specificier til hexadecimal

## Hvad du aldrig må gøre:

- Udskriv bittene i den forkerte rækkefølge (MSB skal være først)
- Glem at shift masken
- Brug division i stedet for bitwise operationer
- Antag at int altid er 32 bits